

Given an alphabet A , we write A^* for the set of all finite strings of any length with letters in A , including the empty string. Thus $\{0,1\}^* = \{\epsilon, 0, 1, 00, 01, 10, 11, 000, 001, \dots\}$, where ϵ denotes the empty string. Given a string w in A^* and a DFA M on A , we let M compute on w as follows: we begin in the start state. Then, for each letter in w , starting from the left we follow the arrow corresponding to that letter from the current state. If the final state we reach when the string ends is an accepting state, then we say that M accepts w . If not, then we say that M rejects w . For instance, if M is the DFA from Figure 1, then we start at state q_0 and move to q_1 on 0, and q_3 on 1. Since q_1 is an accepting state, M accepts w .

WORKSHEET 28

Deterministic Finite Automata and Regular Languages

In this worksheet, we will study a very primitive model of a computer, called a *deterministic finite automaton*, or *DFA* for short. To start with, we need an *alphabet* A , which is a finite collection of symbols we'll call letters. The alphabet could be the usual English alphabet, consisting of 26 letters, for instance, or it could be the binary alphabet, consisting of just two "letters," called 0 and 1. We'll often work with the binary alphabet due to its simplicity.

Given an alphabet A , a DFA on A consists of a directed graph with a finite collection of nodes, called the *states*, with edges labeled by letters in A such that for every state q and every letter a , there is a unique edge out of q labeled with the letter a . Next, one state is designated at the start state, and some states (possibly all of them, possibly none of them), are considered *accepting states*.

We typically represent a DFA by means of a diagram. An example is shown in Figure 1. The states marked with double circles are the accepting states.

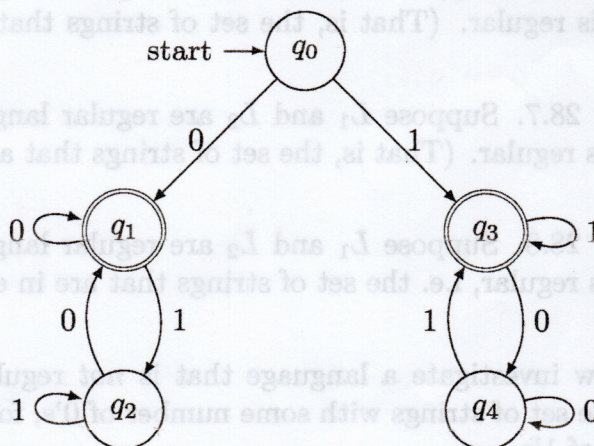


Figure 1. The diagram of a DFA on the alphabet $\{0, 1\}$.

Given an alphabet $\mathcal{A}$, we write $\mathcal{A}^*$ for the set of all finite strings of any length with letters in $\mathcal{A}$, including the empty string. Thus $\{0, 1\}^* = \{\varepsilon, 0, 1, 00, 01, 10, 11, 000, 001, \dots\}$, where ε denotes the empty string. Given a string w in $\mathcal{A}^*$ and a DFA M on $\mathcal{A}$, we let M compute on w as follows: we begin in the starting state. Then, for each letter in w , starting from the left we follow the arrow corresponding to that letter from the current state. If the final state we reach when the string ends is an accepting state, then we say that M *accepts* w . Otherwise, M *rejects* w . For instance, if $w = 0110$ and M is the DFA from Figure 1 then we start at state q_0 and the move to states q_1 , q_2 , q_2 , and q_1 , in that order. Since q_1 is an accepting state, M accepts w .

Given a DFA M , the *language* $\mathcal{L}(M)$ of M is the set of all strings in $\mathcal{A}^*$ that M accepts.

PROBLEM 28.1. For the DFA M above, describe $\mathcal{L}(M)$ concretely, i.e. without reference to a DFA.

PROBLEM 28.2. Find a DFA on $\{0, 1\}$ whose language consists of all finite strings consisting of a bunch of 0's, followed by a bunch of 1's. (That is, no 0 can come after any 1.)

PROBLEM 28.3. Find a DFA on $\{0, 1\}$ whose language consists of all finite strings containing exactly 3 1's, and any number of 0's.

PROBLEM 28.4. Find a DFA on $\{0, 1\}$ whose language consists of all finite strings that contain three 1's in a row somewhere in the string.

PROBLEM 28.5. Find a DFA on $\{0, 1\}$ whose language consists of all finite strings that contain three 1's in a row, but not four 1's in a row.

A language L is said to be *regular* if $L = \mathcal{L}(M)$ for some DFA M .

PROBLEM 28.6. Suppose L_1 and L_2 are regular languages. Show that $L_1 \cup L_2$ is regular. (That is, the set of strings that are in L_1 or L_2 .¹)

PROBLEM 28.7. Suppose L_1 and L_2 are regular languages. Show that $L_1 \cap L_2$ is regular. (That is, the set of strings that are in both L_1 and L_2 .)

PROBLEM 28.8. Suppose L_1 and L_2 are regular languages. Show that $L_1 \triangle L_2$ is regular, i.e. the set of strings that are in exactly one of L_1 and L_2 .

Let us now investigate a language that is *not* regular. Let $L = \{0^n 1^n\}$, i.e. the set of strings with some number of 0's, followed by the same number of 1's.

¹Recall our convention on "or" in mathematics: that includes strings in both L_1 and L_2 .

PROBLEM 28.9. Suppose M is a DFA with s states. Show that if w is a string of length at least $s + 1$, then in the computation of M on w , some state must be visited twice.

PROBLEM 28.10. Suppose state q is visited twice in the computation of M on w . Let x be the substring of w up to the first visit to q , let y be the substring starting after the first visit to q up to the second visit to q , and let z be the substring following the second visit to q . Thus w decomposes as xyz , i.e. first x , then y , then z . Show that M accepts w if and only if it accepts $xyyz$.

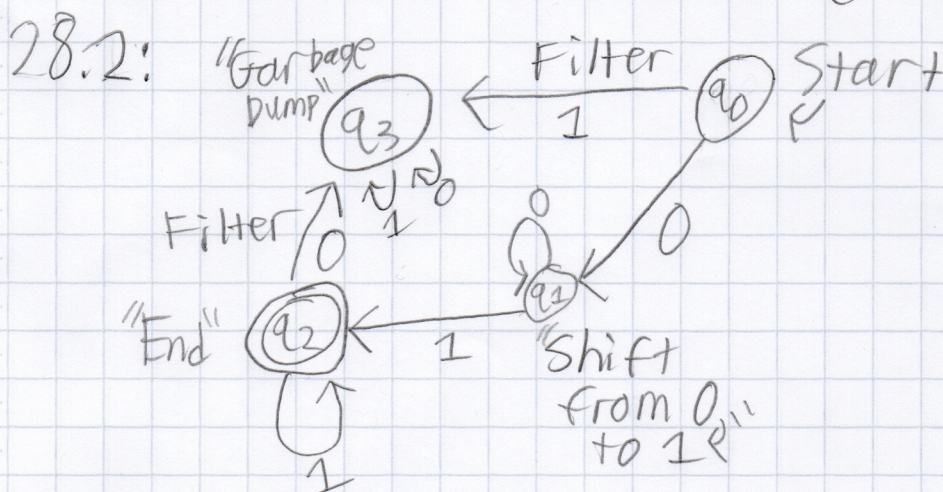
This is a special case of the first significant result in the theory of automata, namely the *pumping lemma*.

PROBLEM 28.11. Show that $L = \{0^n 1^n\}$ is not a regular language.

28.1: Notice that:

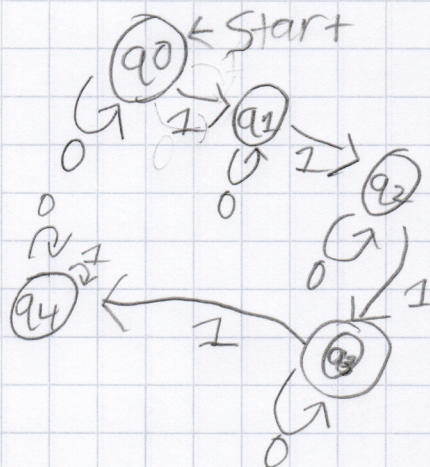
- if $0x_1x_2\dots$ is selected, no matter what the q_1 and q_2 nodes have only 0 or 1 leading into them, respectively.
- if $1x_1x_2\dots$ is selected... q_3 and q_4 nodes have only 1 or 0 leading into them, respectively.

No matter the string, any ending with 0 and starting with 0 (or 1) will lead to q_1 (or q_3).



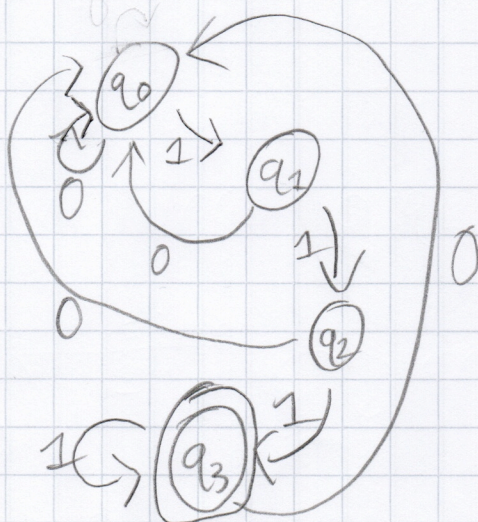
This DFA filters out any early 1s, then (at q_1) decides when the 0 $\rightarrow$ 1 Shift is, then q_2 (the acceptor) filters out any late 0s while keeping 1s.

28.3:



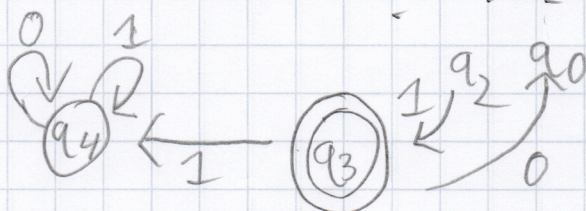
Here, the DFA allows only 3 inputs of 1 before being moved to the "garbage dump" of q_4 .

28.4:



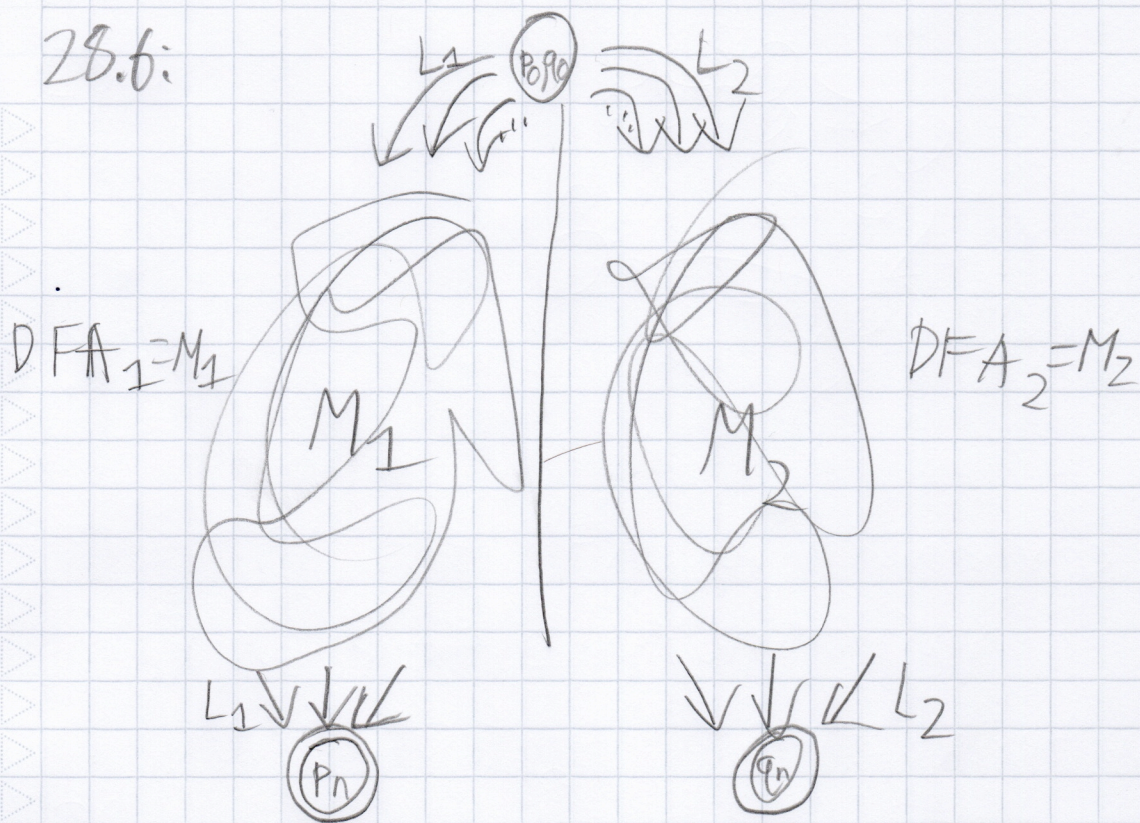
No garbage pile, any string that can achieve three 1's without any 0's between them are accepted.

28.5:



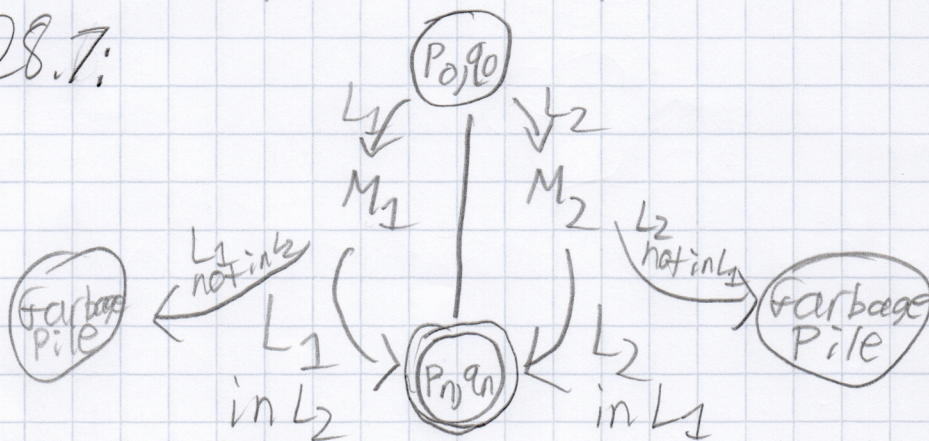
Similar to 28.4, but add the setup shown to prevent four-in-a-row.

28.6:



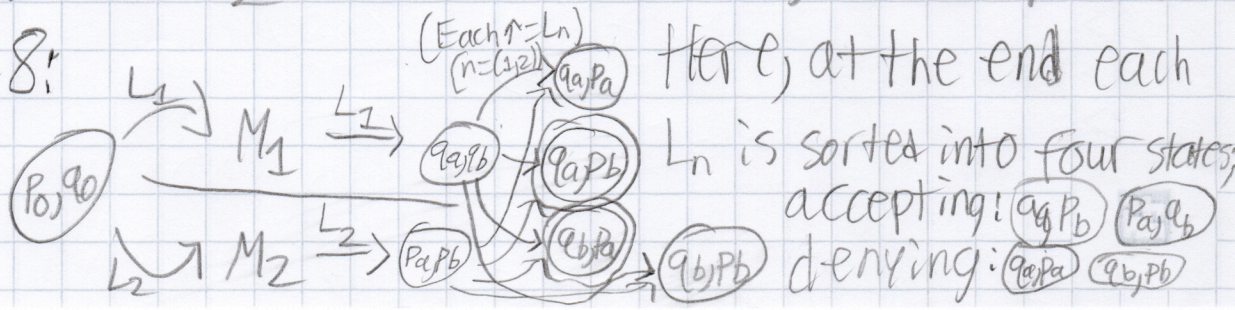
Each language goes through its own machine, connected by a single ordered pair start point for both languages.

28.7:



Here, only the characters in both L_1 and L_2 make it to the (p_n, q_n) acceptor.

28.8:



28.9: Assume that the string never repeats itself at any state. Either way, the string will still double-cross at least one state, because:

- String length $>$ states
- Each state has a unique path

out of it supporting language L_n . Due to the Pigeonhole Principle, it is impossible to visit each state at most once without breaking the above rules of DFAs.

28.10: Say that $x = q_0 \rightarrow q_i$, $y = q_i \rightarrow q_j$, $z = q_j \rightarrow q_n$, and set $q_i = q_j$. If $q_i = q_j$, then $y = q_i \rightarrow q_i$ and $z = q_i \rightarrow q_n$. Because y takes q_i back to itself, y can be repeated as many times as you want before z takes the string to q_n , the accepting state.

28.11: (next page)

* $n \geq 0$

28.11: Assume L is a regular language.

Therefore, a string greater than the pumping length (states in a DFA M that pumps a particular language's # (count)) p in L should satisfy the Pumping Lemma. Call this string $0^p 1^p$.

The PL (Pumping Lemma) states that:

- $0^p 1^p$ can be broken into xy^iz as stated in 28.10.
- y^1 must be within the first p characters of $0^p 1^p$
- If y^1 can be "pumped" i times, $0^p 1^p$ is in L and L is therefore a regular language ($0^p 1^p \in L$).

If y^1 is within the first p characters of $0^p 1^p$, then y^1 must be all 0's.

Pumping y^1 into $y^2, y^3, \dots, y_i$ would create more 0's than ones,

and pumping y^1 into y^0 creates less 0's than ones. This means that

$0^p 1^p$ cannot be pumped. Because

every string longer than p in a regular language must be pumpable, $L = \{0^n 1^n\}$

is NOT a regular language. QED